# AutoSecAudit: An Extensible Multi-Scanner Orchestration Framework with AI-Assisted Remediation Guidance, Precision Fuzzing, and Delta Auditing in DevSecOps

Kodi Vishruth Aithal, Thilak N, Vinaya Kumar B, Vishal Gowda B, Ranjitha R*
*Department of Computer Science and Engineering, Don Bosco Institute of Technology, Bangalore, Karnataka, India*
*Emails: {aithalvishruth, gowdathilakn, bvinaykumar70, vishalgb2005}@gmail.com, *ranjitha.r@dbit.co.in*
*(*Corresponding Author & Faculty Mentor)*

**Abstract**: Automating security verification in fast-paced continuous integration and continuous deployment (CI/CD) pipelines creates operational friction: studies of developer tool adoption report that noisy output and missing actionable guidance discourage the use of automated analysis tools [13]. This paper presents AutoSecAudit, an open, extensible dynamic application security testing (DAST) orchestration framework with remediation guidance, designed for CI/CD pipelines and AI developer tooling. It combines (1) attack-surface discovery through a depth-limited recursive crawler, regular-expression JavaScript route extraction, OpenAPI JSON parsing, and pre-flight reachability checks; (2) concurrent execution of 19 scanner plugins with heuristic false-positive reduction (baseline-response comparison, a two-phase statistical timing check, single-page-application filtering, and adaptive politeness with backoff on HTTP 429/503); (3) an intelligence layer that cross-references findings, enriches CVE-tagged findings with CVSS data, maps findings to OWASP Top 10 (2021), CWE, and PCI DSS v4.0 by keyword rules, and assigns plain-English business-risk categories and a five-level posture grade; and (4) delivery features: predefined multi-framework fix recipes, identifier-based delta comparison between scans, severity gating through exit codes, and a Model Context Protocol (MCP) server with four tools for AI coding assistants. The MCP server supplies scan results and recipes; applying patches is left to the assistant or developer, and closed-loop remediation is not evaluated. In a preliminary single-configuration evaluation on OWASP Juice Shop, eight worker threads reduced scan latency from 184.2 s to 64.8 s (64.8% reduction, 2.84x speedup), and the reported run consolidated 76 raw alerts into 47 findings with precision 97.9% (46/47) and recall 88.5% (46/52) against 52 labelled vulnerabilities. These measurements are author-reported and unreplicated, apply to one benchmark only, and are not evidence of production performance.

**Keywords**: Vulnerability Assessment, DevSecOps, Scanner Orchestration, Model Context Protocol (MCP), AI-Assisted Remediation, Statistical Timing Verification, Compliance Mapping, Delta Auditing, Web Security, API Security.

## I. INTRODUCTION

Modern web applications, cloud-native services, and microservice architectures represent heterogeneous software systems that undergo continuous integration and automated deployment multiple times daily [9]. Ensuring reliable software security within these accelerated delivery cadences motivates 'shift-left' verification, wherein security analysis is executed during automated build and pull-request gating steps [9]. Dynamic Application Security Testing (DAST) evaluates target web services from an external attacker perspective without requiring underlying source code, validating whether identified vulnerabilities are exploitable over active transport protocols.

Embedding DAST in automated build pipelines can be difficult in practice. Johnson et al. [13] report that false positives and poor integration deter developers from using static analysis tools, and Bau et al. [10] found that commercial black-box scanners differ widely in coverage and accuracy. We treat similar friction as a plausible risk for DAST in CI/CD pipelines; this paper does not itself measure developer behavior. We identify five operational challenges that motivate the design:

*1) Heterogeneous Tool Fragmentation and Format Sprawl:* A thorough security audit requires executing multiple specialized utilities—such as port discovery tools (Nmap [1]), web server misconfiguration checkers (Nikto [2]), and API parameter fuzzers. Each utility outputs disparate formats (XML, unstructured console text, non-uniform JSON), forcing engineering teams to write and maintain fragile parsing wrappers.

*2) Duplicate Alert Sprawl and Developer Fatigue:* Independent scanner modules frequently flag the exact same root vulnerability on an endpoint under slightly varied diagnostic titles. Without cross-plugin deduplication, engineers may triage several notifications for a single bug, which contributes to alert fatigue (cf. [13] for static analysis tools).

*3) Executive Communication Gap:* Scanner output is typically expressed as Common Vulnerabilities and Exposures (CVE) identifiers, Common Vulnerability Scoring System (CVSS) vectors, and raw request traces. These technical representations can be hard for executives and product owners to translate into business impact.

 This work is licensed under a Creative Commons Attribution 4.0 International License 

*4) Missing Code Fixes and AI Workflow Disconnection:* Scanners often report what is broken but leave developers to write framework-specific patches, input sanitization routines, or Web Application Firewall (WAF) virtual patching rules themselves. Furthermore, developers increasingly work inside AI-assisted environments (Cursor, Claude Code, Antigravity). Most established scanners were designed around CLI, GUI, or REST interfaces rather than an agent protocol, although Model Context Protocol (MCP) [14] integrations are now emerging for some tools [17], [18].

*5) Stateless Scans Lacking Regression Awareness:* Release gating benefits from knowing what changed between builds. A single scan does not by itself distinguish newly introduced vulnerabilities from previously known ones or show whether a committed patch resolved an open issue; findings-management platforms [19] and scan-comparison features address this in different ways.

To address these challenges, we present AutoSecAudit, an extensible multi-scanner security orchestration framework with remediation guidance for AI-assisted workflows. AutoSecAudit combines automated attack surface discovery, concurrent execution of 19 scanner plugins, false-positive reduction mechanisms, plain-English business danger translation, predefined multi-framework fix recipes, identifier-based delta comparison between scans, and an MCP tool server in one DevSecOps engine. The contributions are the integrated design and its open implementation; the evaluation in Section IV is preliminary.

## II. RELATED WORK AND COMPARATIVE ANALYSIS

Security testing tools generally fit into four architectural paradigms: standalone point scanners, interactive DAST proxies, template-driven matchers, and centralized vulnerability management platforms. Table I summarizes built-in capabilities as described in each tool's public documentation (accessed October 2026) and compares them with AutoSecAudit. Capabilities available only through add-ons, integrations, or configuration are noted as such, and the table is not a head-to-head benchmark.

### A. *Point Scanners and Specialized Utilities*

Network scanners like Nmap [1] handle port discovery and service fingerprinting, while web server checkers like Nikto [2] scan for server misconfigurations and outdated defaults. Template-driven scanners like Nuclei [3] run YAML-defined checks over HTTP and other protocols. These tools are focused scanners: each runs as a standalone binary with its own output format, and none of them by itself correlates results across tools, compares scans across builds, or translates findings into business risk. Nuclei templates do carry severity and classification metadata, and all three tools can emit machine-readable output that external pipelines consume.

### B. *Stateful DAST Proxies and API Fuzzers*

State-aware scanners, explored by Doupé et al. [11], track application navigation states to uncover deeper injection points. In API fuzzing, Atlidakis et al. [12] demonstrated with RESTler that effective testing requires parsing OpenAPI structures. Intercepting proxies and scanners such as OWASP ZAP [4] and PortSwigger Burp Suite provide broad coverage and are considerably more mature than AutoSecAudit. ZAP ships packaged baseline, API, and full scans and an Automation Framework intended for CI use [16]; full active scans with proxy-based tools can be long-running depending on configuration and target, which is why time-boxed or passive modes are common in merge checks. We did not benchmark AutoSecAudit against these tools, so this paper makes no claim of superior coverage or accuracy.

### C. *Vulnerability Management Aggregation Platforms*

Platforms like OWASP DefectDojo [5] aggregate findings from many third-party scanners, deduplicate them, and track their lifecycle, including re-import workflows that close findings absent from a newer report [19]. DefectDojo does not itself execute scans, so teams still need pipeline jobs that run the individual tools and upload their reports. AutoSecAudit instead bundles execution with a lighter-weight post-processing layer; it does not replace a findings-management platform.

### D. *AI Coding Assistants and Agentic Protocols*

Recent advances in AI-assisted software engineering have produced agentic developer tools (such as Cursor, Claude Code, and Antigravity). The Model Context Protocol (MCP) [14] standardizes how such agents call external tools over JSON-RPC 2.0, and MCP integrations now exist for other security tools, including a Burp Suite extension [17] and an alpha-stage ZAP add-on [18]. AutoSecAudit implements an MCP stdio server that lets an agent initiate scans, inspect findings with business-risk annotations, and retrieve predefined fix recipes. The protocol is not a contribution of this work. The server does not edit source code itself, and the agent-driven loop of patching, re-scanning, and checking for regressions is not evaluated in this paper.

 This work is licensed under a Creative Commons Attribution 4.0 International License 

TABLE I COMPARISON OF BUILT-IN CAPABILITIES ACROSS SECURITY TESTING PARADIGMS

| Dimension / Capability | Nmap / Nikto / Nuclei [1-3] | OWASP ZAP [4] | DefectDojo [5] | AutoSecAudit (Ours) |
|---|---|---|---|---|
| Primary role | Focused scanners (ports, web server, templates) | Intercepting proxy and scanner | Findings-management platform; does not run scans | **Plugin-based DAST orchestrator** |
| Parallelism | Tool-specific | Configurable scanner threads | Not applicable | **Plugin-level ThreadPool (default 4, configurable)** |
| Surface discovery | Tool-specific; limited crawling | Spider; OpenAPI import via add-on [16] | Not applicable (imports reports) | **Recursive HTML crawl, regex JS routes; OpenAPI JSON parser (see Sec. III-A)** |
| Scanner breadth | Single purpose or template-defined | Built-in rules plus add-ons | None built in; imports third-party reports | **19 built-in plugins (compact, benchmark-tuned)** |
| False-positive handling | Tool-specific matchers | Rule thresholds and alert filters | Deduplication and triage statuses | **Baseline comparison; 2-phase timing check; SPA filter** |
| Vulnerability intelligence | Template metadata (Nuclei) | Alert references (CWE, OWASP) | Stores imported CVE/CWE/CVSS data | **NVD/CIRCL CVSS lookup for CVE-tagged findings** |
| Compliance mapping | Template tags (Nuclei) | Alert references | Imported metadata and reports | **Keyword rules to OWASP 2021, CWE, PCI DSS v4.0** |
| Business-risk view | Not provided | Risk levels only | Dashboards and metrics | **Plain-English danger categories, 5-level grade (heuristic)** |
| Fix guidance | Tool-specific advice | Per-alert solution text | Imported guidance; Jira integration | **Predefined recipes (Node, Python, Nginx, WAF) and Markdown tickets** |
| AI-agent interface | Not assessed | MCP add-on (alpha) [18] | REST API | **MCP stdio server, 4 tools** |
| Cross-build comparison | Not built in | Not assessed | Re-import closes and reopens findings [19] | **Identifier-based delta (Sec. III-C)** |
| CI/CD gating | Exit status or output parsed by pipeline scripts | Automation Framework exit-status job [16] | Through API and integrations | **--fail-on severity threshold, exit code 1** |

## III. SYSTEM ARCHITECTURE AND METHODOLOGY

AutoSecAudit is structured into four decoupled subsystems (Fig. 1): (1) Input Ingestion, Surface Discovery & Pre-flight Validation; (2) Concurrent Multi-Scanner Engine & Precision Subsystem; (3) Post-Processing Intelligence Layer; and (4) Delivery, CI/CD Gating, and AI-Assistant Integration.

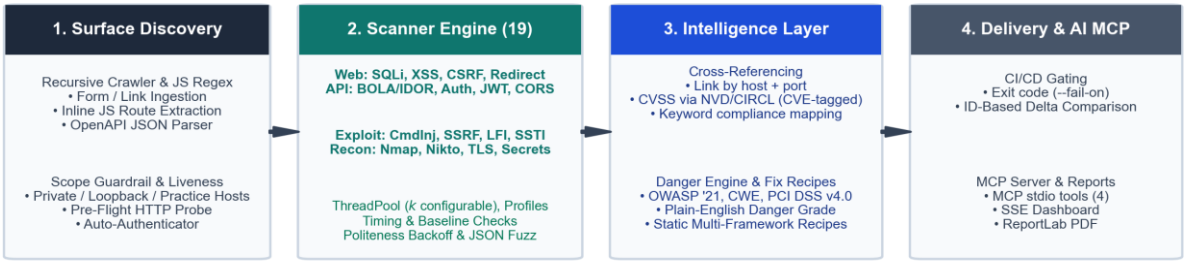



Fig. 1. AutoSecAudit system architecture: four-stage pipeline from surface discovery and concurrent 19-plugin execution through cross-referencing, compliance mapping, danger translation, delta comparison, and MCP integration.

 This work is licensed under a Creative Commons Attribution 4.0 International License 

A. *Attack Surface Discovery and Pre-flight Validation*

Given a target hostname, IP address, or URL, the surface discovery subsystem seeds an endpoint pool E before initiating active scans. The design is motivated by [11], [12] but does not implement state-aware crawling or RESTler-style stateful fuzzing:

*1) Target Scope Guardrail and Pre-Flight Liveness:* Before a scan starts, the function is_target_allowed() accepts only (i) a fixed list of local and intentionally vulnerable practice-target hostnames (for example localhost, juice-shop, dvwa, and webgoat), (ii) hostnames beginning with RFC 1918 prefixes (10., 172.16.-172.31., 192.168.), and (iii) loopback addresses. All other hosts are rejected before any request is sent. This is a guardrail against accidental scanning of public systems, not an authorization mechanism: it does not verify that the operator is permitted to test the target. Matching is lexical rather than resolution-based, so a DNS name that merely starts with a private prefix (for example 10.example.com) or carries an allowlisted suffix is also accepted; address-based validation after DNS resolution is a known gap (Sec. V). The Juice Shop container used in Sec. IV is accepted as a local practice target. In real-scan mode, an HTTP reachability probe (HEAD, falling back to GET, with a raw TCP fallback) confirms that the server responds before plugins are scheduled; the probe is skipped in mock mode.

*2) Recursive Web Crawler and Client-Side JS Route Extraction:* The WebCrawler follows same-host links starting from the root URL, recursing depth-first (not breadth-first) up to a depth limit D_max (default 3) and a page budget (default 20). It extracts HTML form structures (<form action method>), input fields, query parameters, and hyperlinks. Regular-expression extractors are applied to the HTML of each visited page, including inline <script> content, to find client-side REST API routes (/api/v1/...) that lack standard HTML links. Externally referenced JavaScript bundles are not fetched or analyzed in the current implementation, so routes that exist only in such bundles are missed.

*3) OpenAPI / Swagger Ingestion:* For headless REST backends, the OpenAPIImporter reads an OpenAPI 2.0 or 3.x specification supplied as a JSON file or URL and extracts paths, HTTP methods (GET, POST, PUT, DELETE, PATCH), and declared parameters, including path variables. YAML specifications, $ref resolution, and request-body schemas are not handled. In the current CLI and web UI the extracted list is stored on the engine object but is not consumed by the scanner plugins, so OpenAPI ingestion is implemented and covered by a test but not yet integrated into scan-time endpoint seeding.

*4) Automated Multi-Strategy Authenticator:* To audit authenticated application surfaces, the AutoAuthenticator probes common login paths, tries HTML-form, JSON-API, and blind-form login strategies with user-supplied credentials, extracts anti-CSRF tokens, and captures session cookies and bearer tokens. The engine then passes the captured headers and cookies to the crawler and, through set_auth(), to the plugins; we did not audit how much each plugin uses the session. Authentication runs only in real-scan mode.

**Algorithm 1: Attack Surface Discovery (as implemented: depth-first, bounded)**

```
Input: Target URL u0, Max Depth D_max, Page Budget P_max, Auth Credentials A (optional)
Output: Discovered Endpoint Pool E (forms, links, parameters, JS routes)
1: If not IsTargetAllowed(u0): Abort('Target not permitted')
2: If real mode and not CheckLiveness(u0): Abort('Host unreachable')
3: Session S = InitializeSession()
4: If A is provided: S = AutoAuthenticate(u0, A)
5: Visited Set V = {}, E = {}
6: Crawl(u0, 0)
7: Procedure Crawl(u, d):
8:    If d > D_max or |V| >= P_max or Normalize(u) in V: Return
9:    V = V U {Normalize(u)};  Response R = S.GET(u)
10:   If R is not HTML: E = E U {ResourceEndpoint(u)}; Return
11:   E = E U ParseHTMLForms(R.body) U ExtractJSRoutes(R.body)
12:   For each same-host link l in ExtractHyperlinks(R.body):
13:      E = E U {LinkEndpoint(l)}  (with query parameters, if any)
14:      Crawl(l, d+1)
15: Return E
Note: ParseOpenAPISpec(Omega) is a separate importer; its output is not merged into E by
the engine.
```

B. *Core Engine, 19 Scanner Plugins, and Precision Mechanisms*

Each scanner module extends the abstract BaseScanner interface. Standardized findings use the tuple model: F = <id, title, severity, host, port, description, raw_output, c, M>, where c in {low, medium, high} is a heuristic confidence label assigned by the plugin and M holds optional CVE/CVSS, OWASP, CWE, PCI DSS, and remediation fields. The record has no separate endpoint field; endpoint details appear in the title, description, or raw output.

AutoSecAudit organizes 19 scanner plugins across four operational domains:

- Web Application Injections: SQLiScanner (error-based and tautology payloads, plus time-based blind probes verified by the timing check), XSSScanner (reflected XSS only), CSRFScanner (anti-CSRF tokens and SameSite cookie attributes), OpenRedirectScanner (unvalidated redirect parameters), MisconfigScanner (security headers, cookie flags, exposed files, HTTP methods).
- API & Access Control: BOLAIdorScanner (object-identifier manipulation), APIAbuseScanner (excessive data exposure, mass assignment, missing pagination, GraphQL introspection, API versioning, broken authentication), AuthScanner (default credentials, IDOR, weak-password registration, exposed admin panels, user enumeration,

 This work is licensed under a Creative Commons Attribution 4.0 International License 

login rate limiting), JWTScanner (passive inspection of JWTs observed in responses for alg=none headers and missing or expired exp claims; it does not test weak HMAC secrets or forged-token acceptance), CORSScanner (wildcard origin, origin reflection, null origin).

• Advanced Exploit Vectors: CommandInjectionScanner (OS command injection, including timing-verified blind probes), SSRFScanner (internal-URL probes; cloud-metadata probes to 169.254.169.254 are disabled unless AUTOSEC_ENABLE_METADATA_PROBES is set), PathTraversalScanner (local file inclusion, e.g., /etc/passwd), SSTIScanner (arithmetic expression probes such as {{7*7}}, ${7*7}, and <%= 7*7 %>).

• Reconnaissance & Infrastructure: SecretExposureScanner (sensitive paths such as /.env and /.git/config), SSLTLSScanner (certificate validity and expiry, HSTS, obsolete TLS versions), DirBruteScanner (a 50-entry curated path list that includes several Juice Shop-specific entries), NiktoPlugin and NmapPlugin (wrappers around the external nikto and nmap executables, which must be installed).

*Scan Profiles:* The engine supports four scan profiles, defined in config.py and verified by loading each profile: 'full' (all 19 plugins), 'owasp' (11 plugins: SQLi, XSS, CSRF, SSRF, Auth, Misconfig, CommandInjection, PathTraversal, SSTI, SecretExposure, OpenRedirect), 'api' (7 plugins: BOLAIdor, JWT, CORS, APIAbuse, Auth, SQLi, XSS), and 'recon' (6 plugins: SSLTLS, Misconfig, DirBrute, Nikto, Nmap, SecretExposure). The recon profile is labelled 'passive' in the configuration, but it is better described as reconnaissance-oriented: Nmap performs port scanning, and Nikto, DirBrute, Misconfig, and SecretExposure issue active HTTP requests to the target. It is non-exploitative rather than passive. Plugins run concurrently through a thread pool whose size is set by AUTOSEC_THREAD_COUNT (default 4); the evaluation in Sec. IV used 8.

*Precision Mechanisms:* AutoSecAudit includes four mechanisms intended to reduce false positives and target load. They lower, but do not eliminate, false-positive risk:

1) Two-Phase Statistical Timing Check (core/timing.py): Blind time-based probes (for example SLEEP(3) or sleep 2) are prone to false positives from latency spikes. BaseScanner measures m benign requests (m = 2 in the plugin configuration) to estimate a baseline mean mu_base and sample standard deviation sigma_base. For a probe with injected delay tau, phase 1 requires T_delay >= mu_base + rho*tau with rho = 0.70. If phase 1 passes, phase 2 sends a zero-delay control payload; if T_control <= mu_base + 0.35*tau the finding is assigned high confidence, otherwise (or if no control probe is supplied) it is still reported with medium confidence. The decision rule does not use sigma_base, which is recorded for reporting only, and with m = 2 the variance estimate is coarse. The check is therefore a heuristic: a latency spike that persists across both phases can still produce a medium-confidence false positive. It is applied by the SQLi and command-injection plugins only.

2) Baseline Response Comparison: The SQLi plugin requests a benign baseline response and compares each probe response against it; a change in HTTP status or a response-length change above 20% is treated as evidence, graded high, medium, or low by the magnitude of the change. Other plugins use their own matching logic.

3) SPA and Development-Server Filtering (core/spa_detector.py, core/dev_detector.py): Responses that look like single-page-application catch-all pages are filtered to avoid findings caused by soft-404 behavior, and development-server artifacts are tagged.

4) Adaptive Politeness (core/politeness.py): Each plugin adds randomized 0-20 ms jitter between requests. On HTTP 429 or 503 the plugin sleeps for min(2^(n-1), 8) seconds, where n is the count of consecutive throttled responses, before continuing; the throttled response itself is not retried.

A recursive JSON body fuzzer (core/fuzzer.py) enumerates every leaf of a nested JSON request body and replaces one leaf at a time with a payload while preserving the rest of the structure; the SQLi and command-injection plugins use it. Object keys that contain a period are not addressed correctly by its dotted-path scheme, and empty containers have no leaf to mutate.

### C. *Post-Processing Intelligence Layer*

Following raw scanner execution, the intelligence layer post-processes findings in the steps of Algorithm 2, which reflects the implemented CLI pipeline. The cross-referencing step annotates findings that share a host and port but does not merge or remove any of them; a compound-key merge over host, port, endpoint, and CWE, which would be needed to produce the alert consolidation reported in Sec. IV-B, is not part of the released code path (see Sec. IV-B and Sec. V). Compliance mapping applies keyword rules to finding titles to assign OWASP Top 10 (2021) categories [6] and uses a built-in table for CWE identifiers and PCI DSS v4.0 requirements [7]; CVSS data are looked up from the NVD [8] or CIRCL for findings that carry a CVE identifier.

 This work is licensed under a Creative Commons Attribution 4.0 International License 

**Algorithm 2: Intelligence Post-Processing and Delta Comparison (as implemented)**

```
Input: Raw Findings F_raw, optional Previous Report S_{t-1}
Output: Annotated Report S_t, Delta Partition <Delta_NEW, Delta_FIXED, Delta_UNCHANGED>
1: S_t = F_raw
2: Group map G = {}; for each f in S_t: G[<f.host, f.port>] = G[<f.host, f.port>] U {f}
3: For each group g in G with |g| > 1: append 'Related findings: ids' to f.references for
each f in g
4: For each f in S_t with f.cve_id: query NVD (2 s timeout), fall back to CIRCL;
5:    set f.cvss_score if unset and merge references; skip silently on failure
6: For each f in S_t: (f.owasp_tag, f.cwe_id, f.pci_dss) = MapCompliance(f)   [keyword
rules]
7: Danger category/grade and fix recipes are derived from f when a report or MCP response
is built
8: If S_{t-1} is provided, with id(f) the plugin-assigned identifier (f.id):
9:    Delta_NEW = { f in S_t | id(f) not in id(S_{t-1}) }
10:   Delta_FIXED = { f in S_{t-1} | id(f) not in id(S_t) }
11:   Delta_UNCHANGED = { f in S_t | id(f) in id(S_{t-1}) }   (findings with empty id are
ignored)
12: Return S_t, <Delta_NEW, Delta_FIXED, Delta_UNCHANGED>
```

*Plain-English Danger Engine and Executive Posture:* The DangerEngine assigns each finding to one of four business threat categories using keyword rules over the finding title and tool name, with an OWASP-tag fallback: (i) Customer & Database Theft (SQL injection, exposed repositories and environment files, directory listing); (ii) Account Hijacking & Auth Bypass (authentication, JWT, session, cookie, IDOR, and brute-force findings); (iii) Phishing, XSS & Brand Defacement (CORS, XSS, security headers, open redirects); and (iv) Full Server Takeover & RCE (command injection and remote code execution). Unmatched findings default to category (i). The posture score is 100 - (25*C + 12*H + 5*M + 1*L), where C, H, M, and L are the counts of critical, high, medium, and low findings, clamped to the range [10, 100]; together with count thresholds it yields one of five grades (A+, B, C, D, F). The grade and the plain-English explanations (what is broken, what an attacker can do, business impact, estimated fix time) are rule-based heuristics; their usefulness to non-technical readers was not evaluated.

*Fix Recipes and Dev Tickets:* The RecipeEngine returns predefined, static code snippets selected by keywords in the finding title: Node.js/Express (parameterized queries, Helmet, express-validator), Python (SQLAlchemy and Django ORM, Pydantic, Flask/FastAPI middleware, Flask-Talisman), Nginx directives, and a generic WAF rule in JSON form. Recipes exist for seven finding classes (SQL injection, CORS, security headers, directory and secret exposure, CSRF, open redirect, BOLA/IDOR) plus a generic fallback; there are none for XSS, command injection, SSRF, SSTI, JWT, or path traversal. The recipes are templates: they are not generated per codebase, and this paper does not test that applying them removes the corresponding vulnerability. A Markdown ticket template is also provided that can be pasted into issue trackers such as Jira, Linear, or GitHub.

*Identifier-Based Delta Comparison:* The DeltaAnalyzer compares two reports by the finding identifier f.id assigned by the scanner plugin (for example SQLI-001 or AUTH-DEFCRED-001); no content hash is computed. With id(S) denoting the set of identifiers in report S, findings are partitioned into three disjoint sets: Delta_NEW = { f in S_t | id(f) not in id(S_{t-1}) }, Delta_FIXED = { f in S_{t-1} | id(f) not in id(S_t) }, and Delta_UNCHANGED = { f in S_t | id(f) in id(S_{t-1}) }. The rule has edge cases: most plugins build identifiers from a positional counter, so inserting or removing an earlier finding renumbers later ones and can yield spurious NEW/FIXED pairs; the identifier carries no host or endpoint, so two different findings that share an identifier are reported as UNCHANGED (we confirmed both behaviors with small examples); findings with an empty identifier are dropped; and duplicate identifiers within one report collapse. A secondary dictionary-based variant reports only the new and fixed sets.

### D. *AI-Assistant Integration via Model Context Protocol (MCP) and CI/CD Delivery*

AutoSecAudit includes a JSON-RPC 2.0 stdio MCP server (mcp_server.py; protocol revision 2024-11-05) that lets AI developer tools such as Cursor, Claude Code, and Antigravity call the scanner [14]. It implements the initialize, tools/list, tools/call, and ping methods and exposes four tools (Table II). We distinguish four levels of remediation support: (a) generating predefined fix recipes, which is implemented; (b) exposing those recipes to an agent through MCP, which is implemented; (c) an agent applying a change to source code, which is performed by the agent's own editing tools and is outside the server; and (d) re-scanning to show that a change fixed the issue without regressions, which the current verification tool does not do against a live target. Only (a) and (b) are supported by this work.

 This work is licensed under a Creative Commons Attribution 4.0 International License 

TABLE II AUTOSECAUDIT MCP TOOLS AND THEIR CURRENT LIMITATIONS

| MCP Tool Name | Arguments | Behavior as implemented | Limitation in this release |
|---|---|---|---|
| autosec_scan | target (required), profile, mock | Runs a scan and returns grade, severity counts, and the threat matrix. | mock defaults to true, so the default call returns simulated findings. |
| autosec_get_findings | report_id, view | Reads a saved scan report and returns findings with business-danger text and fix recipes. | Reads the key 'findings' while saved reports use 'all_findings'; returned zero findings for a saved report in our check. |
| autosec_get_fix_recipe | finding_title (required), framework | Returns static recipes (nodejs, python, nginx, waf) and a Markdown ticket for a title keyword. | Keyword lookup; seven finding classes plus a generic fallback. |
| autosec_verify_fix | target (required), vulnerability_type (required) | Re-runs a scan and counts findings whose title matches the type; reports RESOLVED or STILL_VULNERABLE. | Scan runs in mock mode, so the live target is not re-tested. |

The server therefore supports recipe delivery and scan access for an AI assistant. It does not edit source files, and this paper does not demonstrate that a recipe or an agent-applied patch removes a vulnerability or avoids regressions; no patch-success rate is reported.

For CI pipelines, the command-line scan accepts --fail-on with the values low, medium, high, or critical. If any finding meets or exceeds the threshold, the process returns exit code 1; it also returns 1 when the target is invalid or unreachable, and 0 otherwise. Progress is streamed to the web dashboard over Server-Sent Events (SSE), and reports can be exported as HTML, JSON, and ReportLab-generated PDF files.

## IV. EXPERIMENTAL EVALUATION AND RESULTS

Evaluation setup. The authors report running AutoSecAudit on an 8-core host with 16 GB RAM running Ubuntu 22.04 LTS against a containerized OWASP Juice Shop instance [15]. The repository's docker-compose file references the image bkimminich/juice-shop:latest, and the exact version tested was not recorded. The run mode is not recorded; real-scan mode is assumed because latency scaling is not meaningful in mock mode. The thread-pool size was set to k through AUTOSEC_THREAD_COUNT. The evaluation addresses four research questions: RQ1 (Latency & Concurrency), RQ2 (Accuracy & Alert Consolidation), RQ3 (Delta Comparison Behavior), and RQ4 (Test Suite Status).

Evidence status. The latency values (Table III) and the detection counts (Table IV) are author-reported values from a single configuration. The raw run logs, the number of repetitions and the aggregation method, the per-finding labels, and the list of 52 ground-truth vulnerabilities are not included in the repository, so these numbers are preliminary and could not be independently verified. Their derived quantities (speedups, percentages, precision, recall, F1) were recomputed and are arithmetically consistent. Each is a point estimate with no confidence interval.

### A. *Concurrency and Execution Latency (RQ1)*

Total audit latency was measured for worker thread counts k in {1, 2, 4, 8, 16}. As shown in Table III and Fig. 2, increasing concurrency from sequential execution (k=1) to k=8 reduced scan time from 184.2 s to 64.8 s, a 64.8% reduction (2.84x speedup; 184.2/64.8 = 2.84). The percentage and the duration in seconds happen to share the value 64.8, but they are different quantities. Performance did not improve beyond k=8. Because concurrency is across 19 plugin tasks, the longest-running plugin bounds the achievable speedup; target-side limits may also contribute, but the cause was not isolated. Whether each table entry is a single run or a mean is not recorded.

TABLE III AUDIT LATENCY ACROSS CONCURRENCY CONFIGURATIONS (AUTHOR-REPORTED)

| Worker Threads (k) | Scan Duration (s) | Speedup | CPU Utilization (%) |
|---|---|---|---|
| 1 (Sequential) | 184.2 s | 1.00x | 14.2% |
| 2 | 102.5 s | 1.80x | 27.8% |
| 4 | 71.4 s | 2.58x | 51.3% |
| **8 (best observed)** | **64.8 s** | **2.84x** | **78.4%** |
| 16 | 66.1 s | 2.79x | 82.1% |

 This work is licensed under a Creative Commons Attribution 4.0 International License 

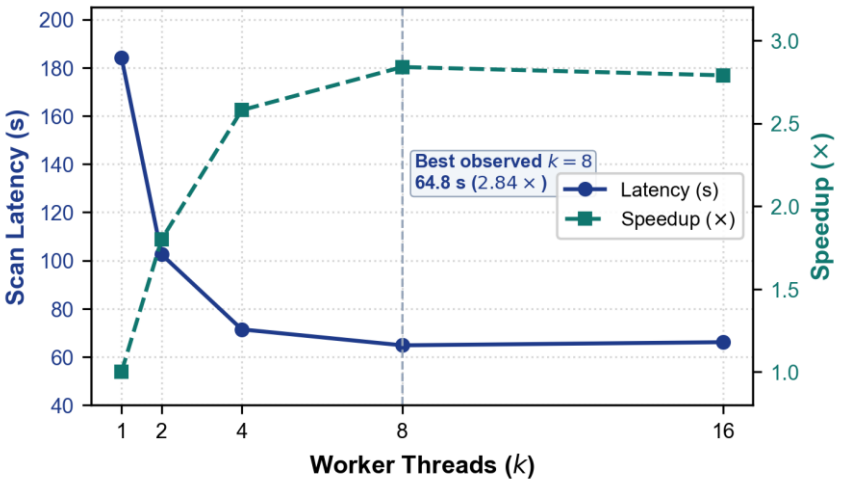



Fig. 2. Audit execution latency and relative speedup across worker thread allocations (k in {1, 2, 4, 8, 16}).

B. *Detection Accuracy and Noise Reduction (RQ2)*

To assess detection accuracy, findings on the benchmark application were compared with Juice Shop's challenge catalog [15]; the 52 in-scope vulnerabilities form the ground truth. A true positive (TP) is a reported finding matched to a ground-truth vulnerability, a false positive (FP) is a reported finding with no match, and a false negative (FN) is a ground-truth vulnerability with no matching finding. Precision = TP/(TP+FP), recall = TP/(TP+FN), and F1 = 2PR/(P+R). Table IV groups the 19 plugins into five evaluation classes; these classes differ from the four functional domains in Sec. III-B (for example, MisconfigScanner appears under Misconfig & Recon here but under web application plugins there). The procedure for matching findings to challenges and for identifying duplicate raw alerts is not documented in the repository, and no second annotator is documented. Several plugin inputs are specific to Juice Shop (for example DirBruteScanner paths such as /ftp and AuthScanner credentials), so the reported precision and recall are benchmark-specific.

TABLE IV VULNERABILITY DETECTION AND CONSOLIDATION METRICS ACROSS 19 PLUGINS (AUTHOR-REPORTED)

| Vulnerability Class (Active Plugins) | True Positives (TP) | False Positives (FP) | Raw Alerts | Consolidated Findings |
|---|---|---|---|---|
| Injection (SQLi, CmdInj, SSTI, OpenRedirect) | 8 | 0 | 14 | 8 |
| Cross-Site Scripting (XSSScanner, CSRFScanner) | 12 | 1 | 19 | 13 |
| Misconfig & Recon (Misconfig, DirBrute, Nikto, Nmap, TLS, Secrets) | 15 | 0 | 26 | 15 |
| Access & Traversal (BOLAIdor, Auth, PathTrav, SSRF) | 6 | 0 | 9 | 6 |
| API & Tokens (APIAbuse, JWT, CORS) | 5 | 0 | 8 | 5 |
| **Total / Summary** | **46** | **1** | **76** | **47 (38.2% fewer than raw)** |

In the reported run, 76 raw alerts were consolidated into 47 findings (46 TP + 1 FP), a 38.2% reduction in alert volume ((76-47)/76; Fig. 3). Against the 52 labelled vulnerabilities, this gives precision 97.9% (46/47), recall 88.5% (46/52, with 6 FN), and F1 92.9% (2*46/(2*46+1+6)). These are results for one benchmark and one run. They do not show that false positives are absent in other applications, and the single false positive shows they are not eliminated even here. Reproducibility caveat: the released Correlator groups findings by host and port and annotates related findings but does not merge them (Sec. III-C), so the 76-to-47 consolidation cannot be reproduced from the released pipeline; the consolidation rule used in the reported run should be documented and released.

 This work is licensed under a Creative Commons Attribution 4.0 International License 

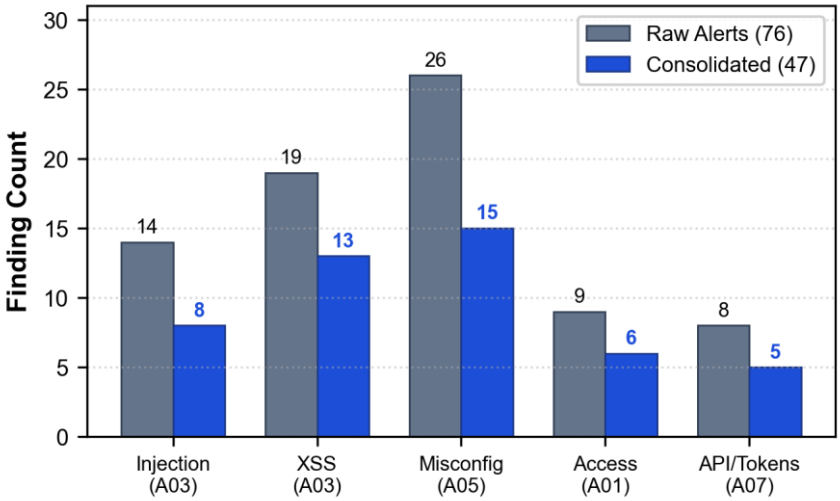



Fig. 3. Raw alerts vs. consolidated findings by evaluation class (Table IV; 76 to 47, a 38.2% reduction in alert volume).

C. *Delta Comparison Behavior (RQ3)*

Delta comparison was exercised on a three-build synthetic sequence (Fig. 4): (1) Build v1.0 (baseline): 47 findings; (2) Build v1.1: 10 findings removed and 3 added, giving 3 Delta_NEW, 10 Delta_FIXED, and 37 Delta_UNCHANGED (47 - 10 + 3 = 40 active findings); and (3) Build v1.2: 14 findings removed, giving 0 Delta_NEW, 14 Delta_FIXED, and 26 Delta_UNCHANGED (40 - 14 = 26 active findings). We re-ran the repository's DeltaAnalyzer on identifier sets constructed to match these counts (paper/revision_audit/verify_delta_sequence.py), and it produced the same partitions. Because the identifier sets were constructed by hand, this confirms only that the set arithmetic is implemented as described for this one sequence; it is not evidence of reliability on real builds. As Sec. III-C explains, identifier-based matching misclassifies renumbered findings and findings that share an identifier, so no general fidelity figure is claimed.

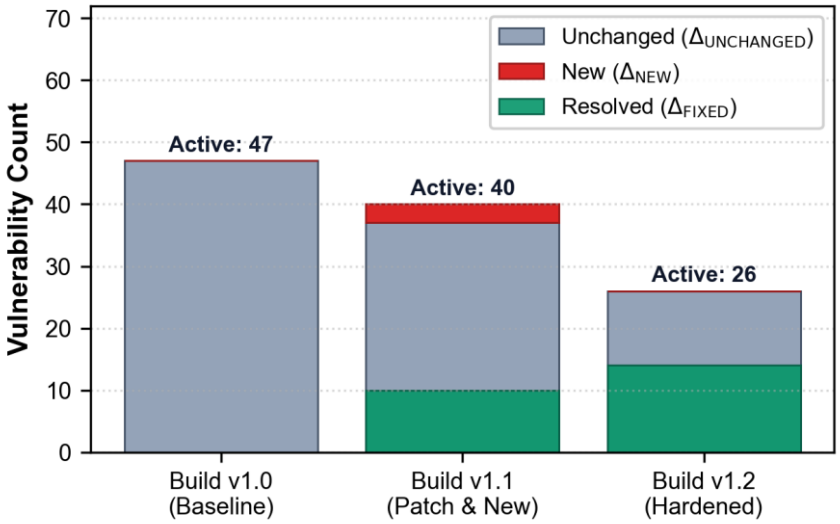



Fig. 4. Delta partitions for the synthetic three-build sequence (counts constructed to match Sec. IV-C).

D. *Test Suite Status (RQ4)*

The repository contains 165 tests collected by pytest. In our verification run, 164 passed and 1 failed: test_auth.py asserts that registering the user 'testdev' succeeds, and it failed because that account already existed in the persistent user database, so the test is not idempotent. The project's unittest runner (run_tests.py) discovers 161 tests, all of which passed; pytest additionally collects function-style tests that unittest discovery does not. The tests cover plugin behavior (mostly in mock mode), the timing engine, the JSON fuzzer, the danger engine and recipes, MCP message handling, scan profiles, target reachability, and the --fail-on exit code. Mock-mode tests check parsing and control flow, not detection accuracy against live targets, and no test covers the 47-40-26 delta sequence or the 76-to-47 consolidation.

## V. DISCUSSION, LIMITATIONS, AND DEPLOYMENT CONSIDERATIONS

Deployment considerations. (1) Single-page applications: regular-expression extraction finds API routes embedded in the HTML of visited pages, but externally loaded JavaScript bundles are not analyzed; pairing the crawler with a headless browser (for example Playwright) would be needed for full DOM-driven discovery. (2) Offline operation: CVSS

 This work is licensed under a Creative Commons Attribution 4.0 International License 

enrichment applies only to findings that carry a CVE identifier, uses 2 s network timeouts, and is skipped silently when NVD and CIRCL are unreachable; there is no embedded offline CVE dictionary. The CWE and PCI DSS tables used by the compliance mapper are built in. (3) Enterprise authentication: SAML 2.0 and mTLS flows are not supported by the automatic authenticator; pre-authenticated headers can be supplied with the --headers option. (4) Default mode: the configuration default is mock mode (AUTOSEC_MOCK_MODE=true), which returns simulated findings, so real scans must be requested explicitly. (5) Transport: scanner requests disable TLS certificate verification.

Limitations and threats to validity. The evaluation uses one benchmark, one host, and one configuration, and the measurements are author-reported without retained raw logs or repetitions, so no variance can be given. Some plugins contain Juice Shop-specific paths and credentials, which favors the benchmark. Ground-truth labelling was not performed by an independent annotator. The 76-to-47 consolidation depends on a merge rule that the released Correlator does not implement. The delta comparison relies on plugin-assigned identifiers and is fragile when identifiers shift. The target guardrail is lexical and is not an authorization control. The MCP findings-retrieval tool returned no findings for saved reports because of a field-name mismatch, and the verification tool does not re-test a live target. OpenAPI endpoints are parsed but not yet fed to the plugins, the JWT plugin inspects tokens passively, and the recon profile contains active probes. No head-to-head comparison with ZAP, Burp Suite, Nuclei, or other tools was performed, and the fix recipes and danger grades were not validated with developers or executives.

Recommended follow-up work. (a) Repeat the k-sweep at least ten times per setting with a pinned Juice Shop version and report medians and dispersion. (b) Publish the ground-truth list, per-finding labels, and raw alerts, ideally with two independent annotators. (c) Implement and evaluate an explicit consolidation rule such as <host, port, normalized endpoint, CWE>. (d) Replace positional identifiers with a content-derived key and test renumbering cases. (e) Fix the MCP findings key mismatch and make verification run against the live target. (f) Run an agent-in-the-loop study that applies recipes, re-scans, and checks for regressions. (g) Compare against ZAP baseline and API scans on the same target.

## VI. CONCLUSION AND FUTURE WORK

AutoSecAudit is an open, extensible DAST orchestration framework that runs 19 scanner plugins concurrently, adds heuristic false-positive reduction, cross-references and annotates findings with compliance and business-risk context, offers predefined fix recipes, and exposes scanning and recipe lookup to AI coding assistants through an MCP server. In a preliminary single-configuration evaluation on OWASP Juice Shop, eight worker threads reduced scan latency by 64.8% (2.84x speedup) relative to sequential execution, and the reported run yielded 97.9% precision and 88.5% recall on the labelled vulnerabilities with 38.2% fewer alerts after consolidation. These measurements are author-reported, were not independently replicated, and apply to this benchmark only; the consolidation step is not reproducible from the released code path. The delta comparison reproduced the expected partitions on a synthetic identifier sequence but depends on plugin-assigned identifiers. Closed-loop AI remediation, in which an agent applies a patch and a live re-scan confirms the fix without regressions, remains future work. Further work will address the limitations in Sec. V, evaluate local LLM agents for patch synthesis, and explore distributed multi-node scanning.

## ACKNOWLEDGMENT

This research was conducted as part of the AutoSec Security Engineering Initiative in the Department of Computer Science and Engineering at Don Bosco Institute of Technology, Bangalore. The authors express sincere gratitude to faculty mentor Ms. Ranjitha R, department leadership, and the open-source security engineering community for foundational tools, benchmark platforms, and ongoing support.

## REFERENCES

[1] G. F. Lyon, Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Vulnerability Scanning. Insecure.Com LLC, 2009.
[2] C. Sullo and D. Lodge, "Nikto Web Server Scanner," Open Source Security Tool, 2023. Available: https://cirt.net/Nikto2
[3] ProjectDiscovery, "Nuclei: Fast and Customizable Vulnerability Scanner Based on Simple YAML DSL," 2024. Available: https://github.com/projectdiscovery/nuclei
[4] OWASP Foundation, "OWASP Zed Attack Proxy (ZAP)," 2024. Available: https://www.zaproxy.org/
[5] OWASP Foundation, "OWASP DefectDojo: Vulnerability Management Orchestration," 2024. Available: https://www.defectdojo.org/
[6] OWASP Top 10 Team, "OWASP Top 10: 2021 The Ten Most Critical Web Application Security Risks," OWASP Foundation, 2021. Available: https://owasp.org/Top10/
[7] PCI Security Standards Council, Payment Card Industry Data Security Standard (PCI DSS): Requirements and Testing Procedures, Version 4.0, 2022.
[8] National Institute of Standards and Technology (NIST), "National Vulnerability Database (NVD) REST API v2.0," U.S. Department of Commerce, 2023.
[9] M. Myrbakken and R. Colomo-Palacios, "DevSecOps: A Multivocal Literature Review," in Communications in Computer and Information Science, vol. 770, Springer, 2017, pp. 17–29.
[10] J. Bau, E. Bursztein, D. Gupta, and J. C. Mitchell, "State of the Art: Automated Black-Box Web Application Vulnerability Testing," in Proc. 31st IEEE Symposium on Security and Privacy (S&P '10), 2010, pp. 332–345.
[11] A. Doupé, M. Cova, and G. Vigna, "Enemy of the State: A State-Aware Black-Box Web Vulnerability Scanner," in Proc. 21st USENIX Security Symposium (USENIX Security '12), 2012, pp. 523–538.
[12] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API Fuzzing," in Proc. 41st IEEE/ACM International Conference on Software Engineering (ICSE '19), 2019, pp. 748–758.
[13] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why Don't Software Developers Use Static Analysis Tools to Find Bugs?" in Proc. 35th IEEE/ACM International Conference on Software Engineering (ICSE '13), 2013, pp. 672–681.
[14] Anthropic, "Model Context Protocol (MCP) Specification," 2024. Available: https://modelcontextprotocol.io
[15] OWASP Foundation, "OWASP Juice Shop," project page. Available: https://owasp.org/www-project-juice-shop/

 This work is licensed under a Creative Commons Attribution 4.0 International License 

[16] ZAP Project, "Automation Framework," ZAP documentation. Available: https://www.zaproxy.org/docs/automate/automation-framework/
[17] PortSwigger, "Burp Suite MCP Server extension," GitHub repository. Available: https://github.com/PortSwigger/mcp-server
[18] ZAP Project, "MCP Integration add-on," ZAP documentation. Available: https://www.zaproxy.org/docs/desktop/addons/mcp-integration/
[19] DefectDojo, "About Deduplication" and "Reimport," DefectDojo documentation. Available: https://docs.defectdojo.com/

## BIOGRAPHY

The authors are undergraduate research scholars and faculty mentors in the Department of Computer Science and Engineering at Don Bosco Institute of Technology (DBIT), Bangalore, affiliated with Visvesvaraya Technological University (VTU), Belagavi, Karnataka, India. Their research focuses on automated vulnerability assessment, DevSecOps engineering, intelligent security orchestration, and agentic AI remediation systems.

 This work is licensed under a Creative Commons Attribution 4.0 International License 